Task 1:

Here, we'll practice working with *escape sequences* to represent special text. In particular, we'll see how to use the special string "\n" to represent a *new line*. Write a function, `format_address(name, street, city, state, zip_code)` which creates a formatted address block from the five parts of an address. Your function should *return* the formatted address. The expected format is:

Name
Street
City, State ZIP

Note the new lines after Name and Street, and the comma after City.

The signature of this function is `(string, string, string, string, string) --> string`.

Task 2:

Here, we'll see how to use the `len()` function, and work with a couple of string methods. Write a function, `mangle_text(some_string)`. Your function should:

- Take the first half of the string and convert it to lowercase
- Take the second half of the string and convert it to uppercase
- Concatenate in opposite order (second half first, then the first half)
- Return the concatenated string

For example, `mangle_text("SomeText")` should return `TEXTsome`

The signature of this function is `(string) --> string`.

Task 3:

For this task, we'll write a function called `find_emails(text)` to identify and print email addresses found in the `text`. We'll use a simple rule for identifying email addresses – an email is any sequence of characters, separated by spaces, that contains both an at sign (`@`) and a period (`.`). So, things like bob@email.com fit our rule of what an email address looks like. Note that a string might have *multiple* email addresses in it, such as in the following string: *"Contact us at info@example.com or support@company.com for assistance."*

The signature of this function is `(string) --> None`.

Task 4:

This task is a buy-one-get-one-free deal, where we'll write *two* functions!

The first function, `roman(s)`, will take a single Roman numeral character as input and return its corresponding integer value. The Roman numeral system follows specific values: `'I'` is 1, `'V'` is 5, `'X'` is 10, `'L'` is 50, `'C'` is 100, `'D'` is 500, and `'M'` is 1000. If the input is not a valid Roman numeral character, the function should return 0. For example, `roman('I')` should return 1, and `roman('Z')`, an invalid input, should return 0.

The second function, `romanToInt(s)`, should take a string of Roman numerals (like `'XIII'` or `'IV'`) and convert it to its integer equivalent. Roman numerals are usually added together, **but** there are special cases where subtraction is used: for example, `'IV'` is 4 (5 - 1) and `'IX'` is 9 (10 - 1). The general rule for subtraction is that when a smaller numeral precedes a larger numeral, it is subtracted from the larger numeral (e.g., `'IV'` = 4 and `'IX'` = 9). The function needs to account for both the addition and subtraction rules. For example, `romanToInt('XIII')` should return 13, `romanToInt('IX')` should return 9, and `romanToInt('MCMXCIV')` should return 1994. You can assume that inputs are always valid arguments.

These two functions will help you work with Roman numerals, handling both single-character conversions and full numeral strings.

The signature of both functions is the same, `(string) --> int`.

Task 5:

Another buy-one-get-one free deal! Here, we'll write two functions to practice string manipulation.

The first function, `removePunctuations(text)`, will take a string as input and remove the punctuation marks `?!.,;:"'`.

For example, the input `"Hello, world!"` should return `"Hello world"`, and `"Python's fun, right?"` should return `"Pythons fun right"`.

Make sure to test your function with different sentences containing punctuation to ensure it works as expected.

The signature of this function is `(string) --> string`.

Next, we'll build a function, `find_and_replace(text, old_word, new_word, case_insensitive=True, word_boundary=True)`. This function will replace occurrences of `old_word` in the text with `new_word`.

However, it's a bit more complicated than that – you can't just return `text.replace(old_word, new_word)`.

First, we want to support case-insensitive replacements. If case_insensitive=True, replacing "fox" with "wolf" in the sentence "The quick brown Fox jumps over the lazy dog." should return "The quick brown wolf jumps over the lazy dog."

Next, if you set `word_boundary=False`, replacing "fox" in "foxes" should yield "wolves". Similarly, replacing "fox" in "fox!" should replace the word without affecting the punctuation.

Note: This task is *tricky!* I would strongly encourage you to build up a solution a bit at a time – first, just handle replacing words with no special handling of these additional requirements. Next, add support for case-insensitive replacement. Then, try to get the `word_boundary` behaviour to work properly. Be sure to test different combinations of input, considering punctuation and different cases, and don't feel discouraged if you don't get it all working perfectly.

Examples:

```
text = "The quick brown fox jumps over the lazy dog."
    old_word = "fox"
    new_word = "wolf"
```

```
    print(find_and_replace_advanced(text, old_word, new_word,
case_insensitive=True, word_boundary=True))
    # output: The quick brown wolf jumps over the lazy dog.

    text = "The quick brown Fox jumps over the lazy dog."
    old_word = "fox"
    new_word = "wolf"
    print(find_and_replace_advanced(text, old_word, new_word,
case_insensitive=False, word_boundary=True))
    # output: The quick brown Fox jumps over the lazy dog.


    text = "The quick brown fox, jumps over the lazy dog."
    old_word = "fox"
    new_word = "wolf"
    print(find_and_replace_advanced(text, old_word, new_word,
case_insensitive=True, word_boundary=True))
    # output: The quick brown wolf, jumps over the lazy dog.


    text = "The foxes are clever."
    old_word = "fox"
    new_word = "wolf"
    print(find_and_replace_advanced(text, old_word, new_word,
case_insensitive=True, word_boundary=True))
    # output: The foxes are clever.


    text = "The foxes are clever."
    old_word = "fox"
    new_word = "wolf"
    print(find_and_replace_advanced(text, old_word, new_word,
case_insensitive=True, word_boundary=False))
    # output: The wolfes are clever.


    text = "The fox! ran away."
    old_word = "fox"
    new_word = "wolf"
    print(find_and_replace_advanced(text, old_word, new_word,
case_insensitive=True, word_boundary=True))
    # output: The wolf! ran away.


    text = "The fox and the Fox were friends."
```

```
    old_word = "fox"
    new_word = "wolf"
    print(find_and_replace_advanced(text, old_word, new_word,
case_insensitive=True, word_boundary=True))
    # output: The wolf and the wolf were friends.


    text = "The fox ran after the rabbit. Another fox watched."
    old_word = "fox"
    new_word = "wolf"
    print(find_and_replace_advanced(text, old_word, new_word,
case_insensitive=True, word_boundary=True))
    # output: The wolf ran after the rabbit. Another wolf watched.


    text = ""
    old_word = "fox"
    new_word = "wolf"
    print(find_and_replace_advanced(text, old_word, new_word,
case_insensitive=True, word_boundary=True))
    # output: empty string


    text = "The cat runs away."
    old_word = "fox"
    new_word = "wolf"
    print(find_and_replace_advanced(text, old_word, new_word,
case_insensitive=True, word_boundary=True))
    # output: The cat runs away.
```